MASTER OF SCIENCE
IN ENGINEERING

Adaptive filter

Statistical Signal Processing and Modelling

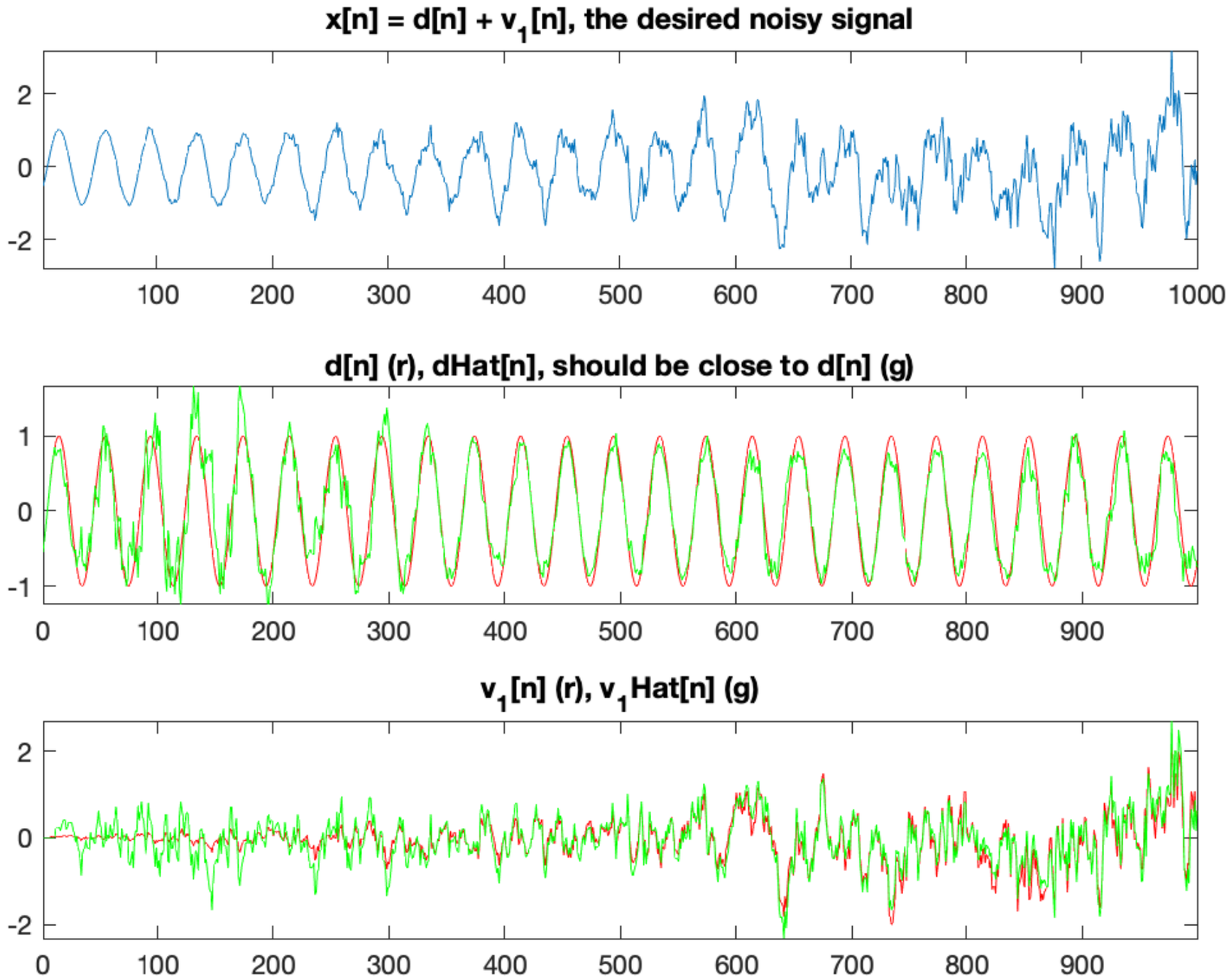
Lavanchy David (HEIG-VD)
Tognolini Maurizio (HEIG-VD)

Plan

- Introduction
- FIR Adaptive filter
- Recursive Least Square

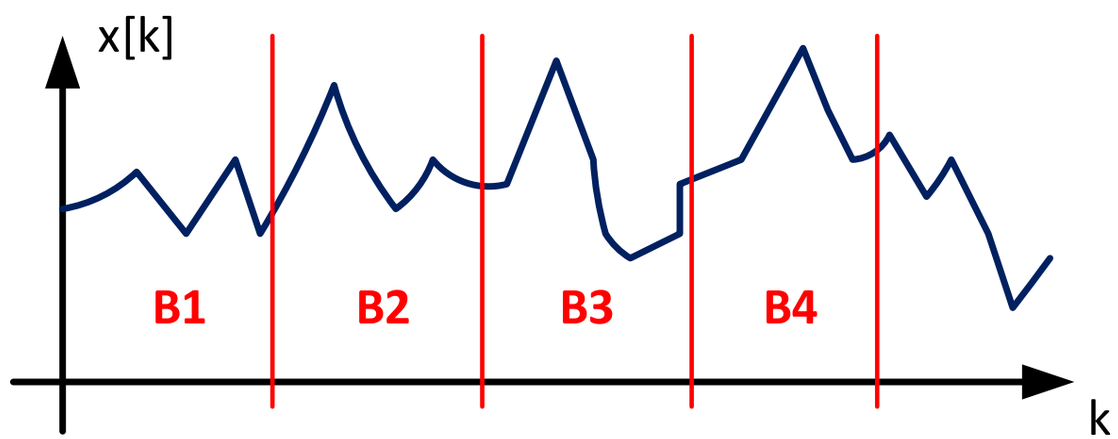
Introduction

- Noise cancellation of non stationary noise



Introduction

- **In previous**, for signal modeling, Wiener filtering and spectrum estimation, **signals were stationary**
- In most application will be **non-stationary**
- One way is **to process in blocks**, where **we suppose stationary**.



- This approach is limited for several reasons:
  - For rapid varying process, the interval will be too small
  - Not easily accommodate step change within intervals
  - Imposes an incorrect model of the data
- **A better approach is proposed with adaptive filtering**

Introduction

- Let's remind Wiener filter for WSS process

$$\hat{d}[n] = \sum_{k=0}^{p-1} w[k] \cdot x[n-k]$$

- The filter coefficients $w[k]$ that minimize the modeling error $e[n] = d[n] - \hat{d}[n]$ in the mean square sense $\mathrm{E}\{|e[n]|^2\}$ are found by solving the Wiener-Hopf equation

$$\boldsymbol{R}_{xx} \cdot \boldsymbol{w} = \boldsymbol{r}_{dx}$$

- However, if $d[n]$ and $x[n]$ are **non-stationary,** then the filter coefficients that minimize $\mathrm{E}\{|e[n]|^2\}$ will depend on n.

$$\hat{d}[n] = \sum_{k=0}^{p-1} w_n[k] \cdot x[n-k]$$

$$\hat{d}[n] = \boldsymbol{w}_{\boldsymbol{n}}^{\boldsymbol{T}} \cdot \boldsymbol{x}[n]$$

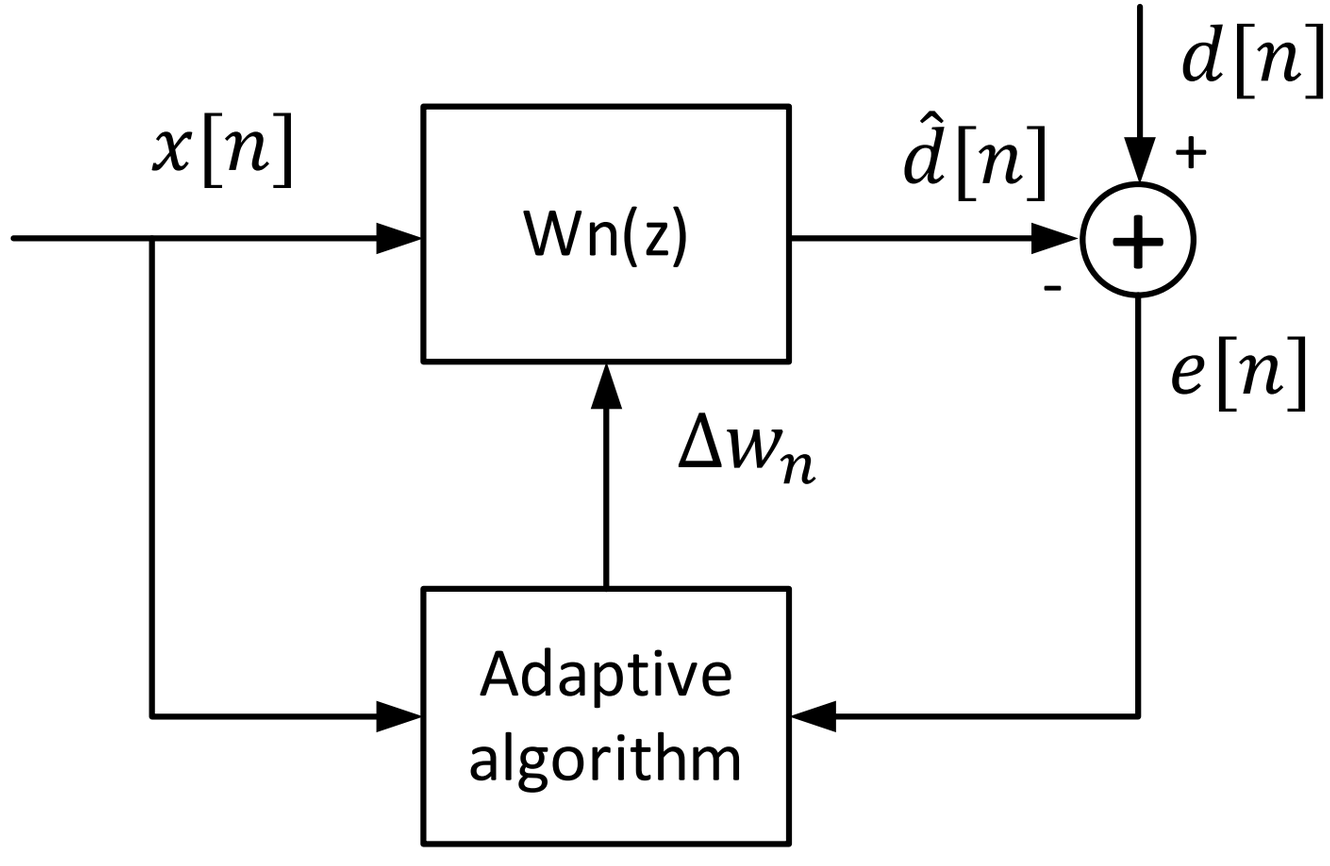
- Minimizing $\mathrm{E}\{|e[n]|^2\}$ for each n will be very difficult.

Introduction

- Problem may be simplified if we relax the requirement that $\boldsymbol{w}_n$ minimize the mean-square error at each time n and consider, instead, a coefficient update equation of the form

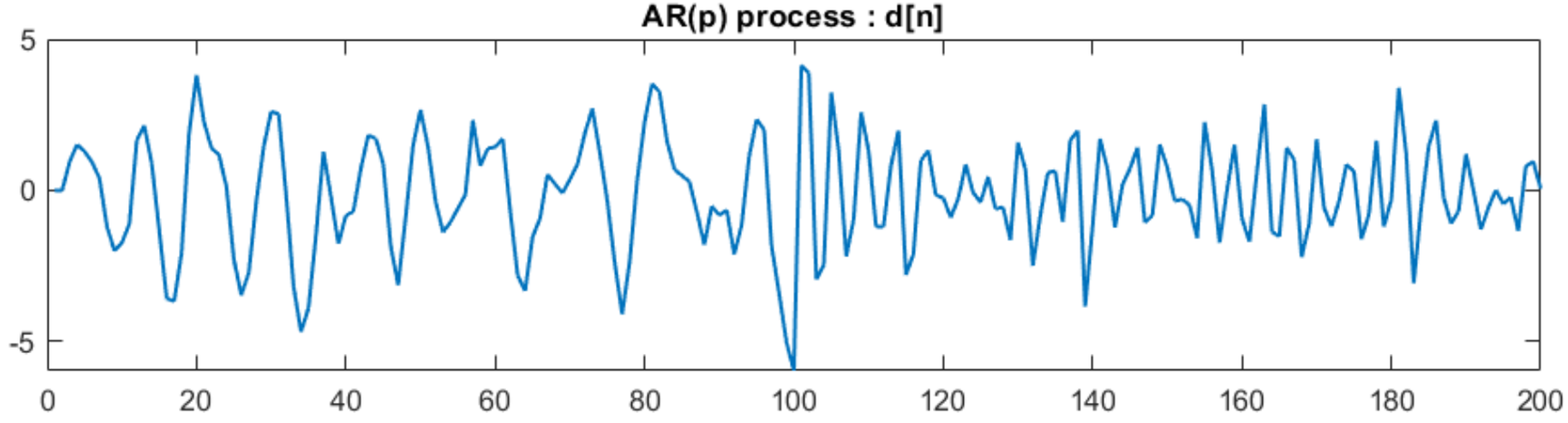
$$\boldsymbol{w}_{n+1} = \boldsymbol{w}_n + \Delta\boldsymbol{w}_n$$

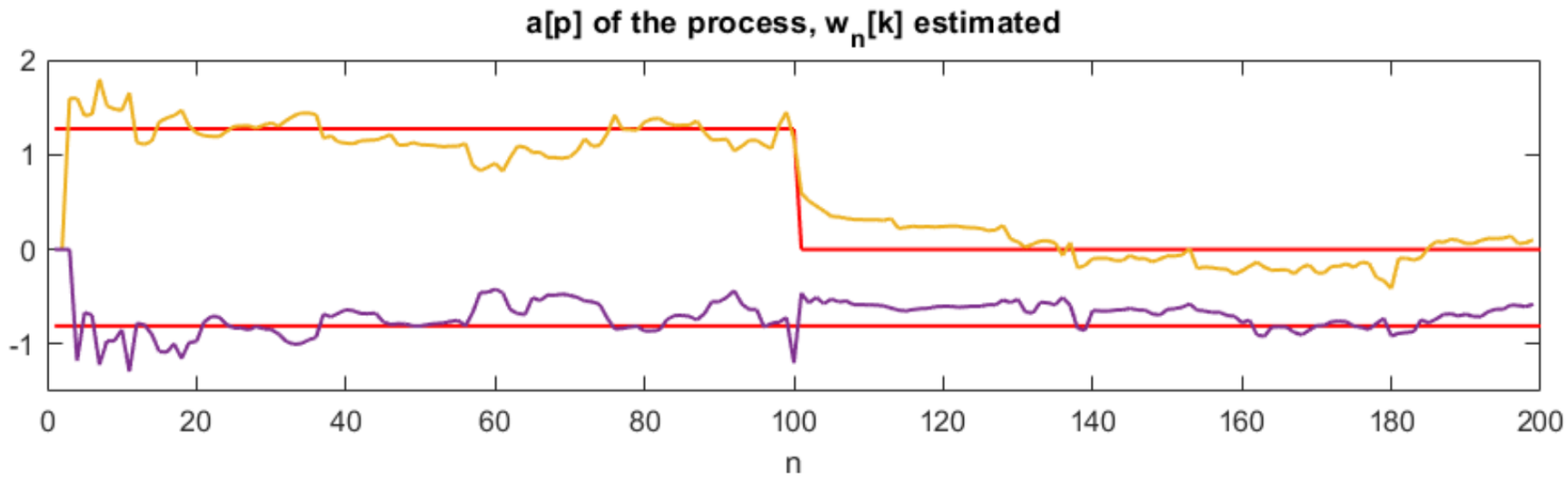
Correction coefficient



Introduction

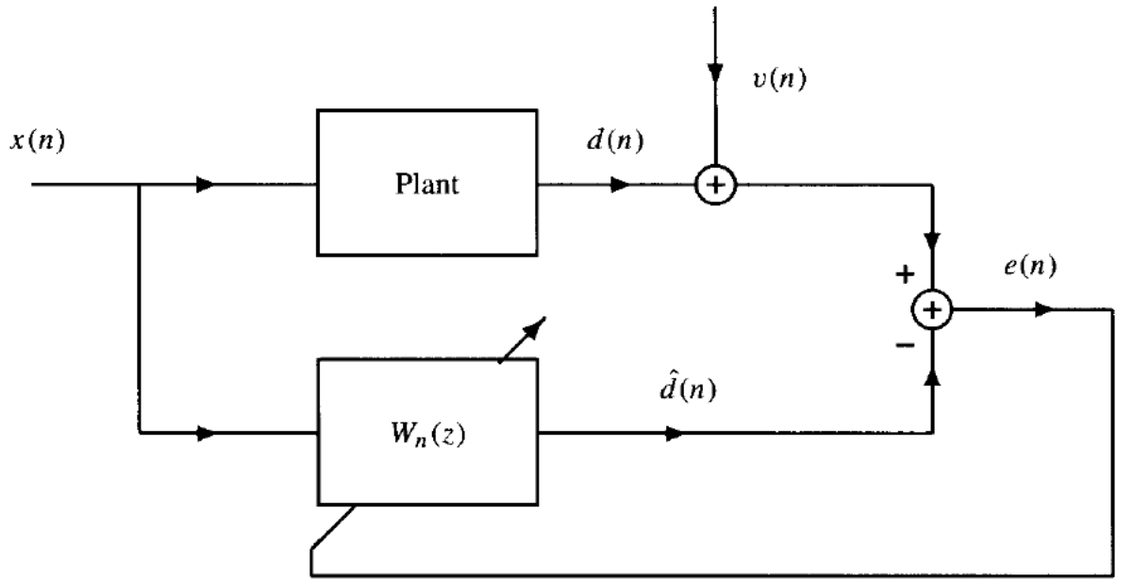
- Adaptive algorithm should
  1. For stationary process, correction should make $\boldsymbol{w}_n$ converge to the Wiener-Hopf equation : $\lim_{n\to\infty} \boldsymbol{w}_n = \boldsymbol{R}_{xx}^{-1} \cdot \boldsymbol{r}_{dx}$
  2. Don't need to know the statistics $r_{xx}[k]$ and $r_{dx}[k]$ to compute $\boldsymbol{\Delta w}_n$
  3. For non-stationary signals, the filter should be able to adapt to the changing statistics and "track" the evolution as it evolves in time.



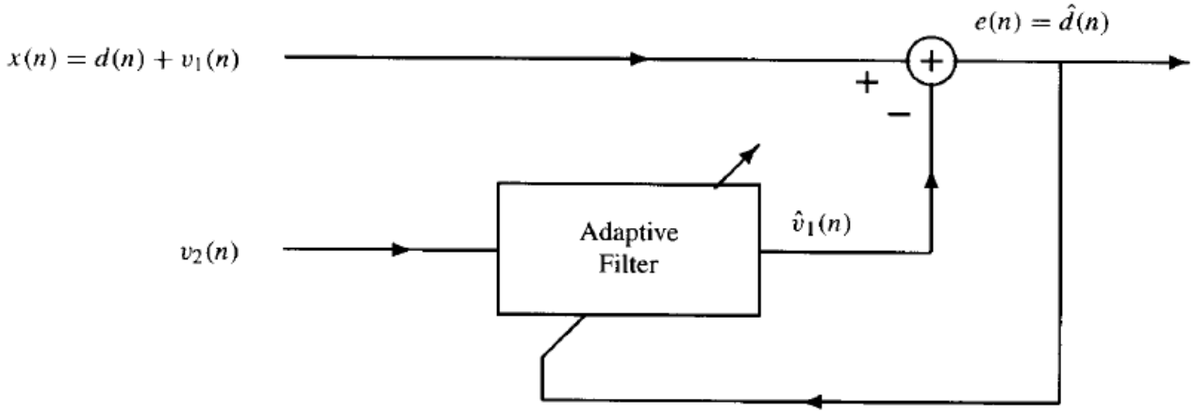


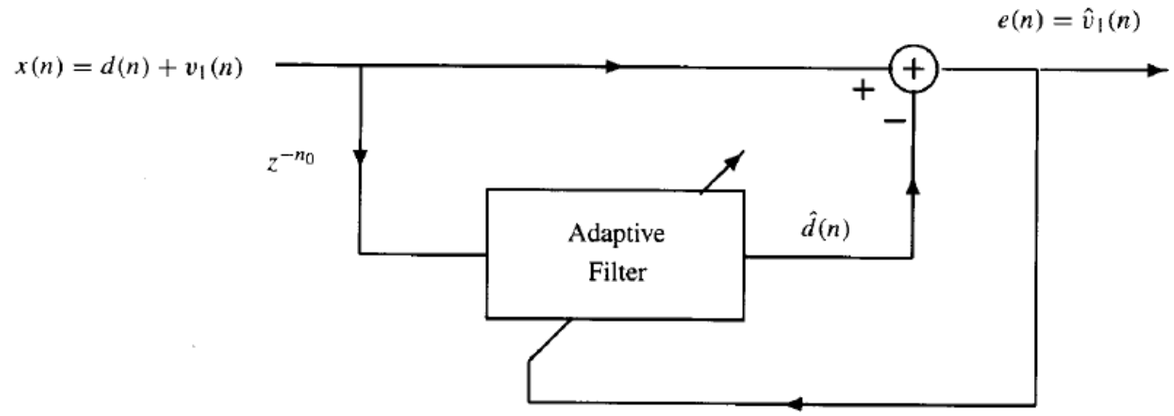
Error computation

- System identification



- Noise cancellation





Plan

- Introduction
- FIR Adaptive filter
- Recursive Least Square

Steepest descent

- Goal is to find $\boldsymbol{w}_n$ at time n that minimizes: $\xi[\mathrm{n}] = \mathrm{E}\{|e[n]|^2\}$

- Wiener-Hopf equation is the solution of : $\frac{\partial \xi[\mathrm{n}]}{\partial w_k} = 0$

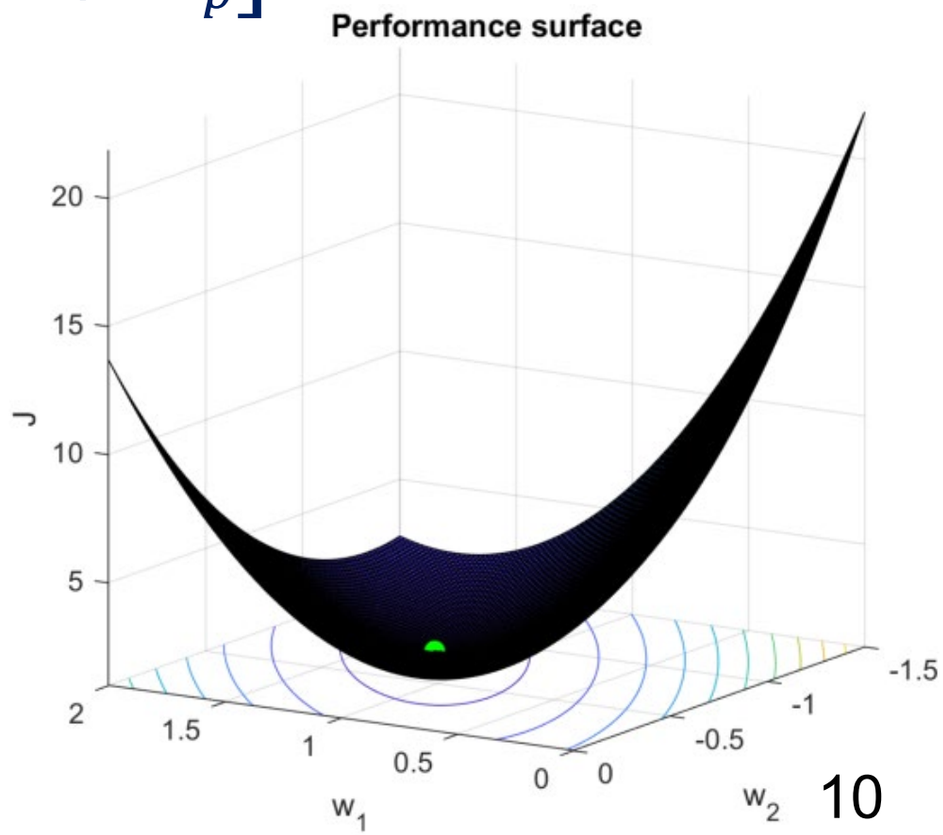
- The gradient vector is : $\nabla \xi[\mathrm{n}] = \begin{bmatrix} \partial \xi[\mathrm{n}] / \partial w_1 \\ \vdots \\ \partial \xi[\mathrm{n}] / \partial w_p \end{bmatrix}$

- Thus, the update equation is :

$$w_{n+1} = w_n - \mu \cdot \nabla \xi[\mathrm{n}]$$

Negative because opposite direction of the gradient

Step size



Steepest descent

- The iterative algorithm that minimize $\xi[n]$ is as follows:

1. Initialize the algo with an initial estimate $\boldsymbol{w}_0$
2. Evaluate the gradient of $\xi[n]$ at the current estimate $\boldsymbol{w}_n$
3. Update the estimate at time n by adding a correction:

$$\boldsymbol{w}_{n+1} = \boldsymbol{w}_n - \mu \cdot \nabla\xi[n]$$

4. Go back and repeat 2 and 3.

Convergences

- Let us evaluate the gradient vector : $\nabla\xi[\mathrm{n}]$ (w_n is complex):

$$\nabla\xi[\mathrm{n}] = \nabla E\{|e[n]|^2\} = E\{\nabla|e[n]|^2\} = E\{e[n]\cdot \underbrace{\nabla e^*[n]}_{\text{-}\boldsymbol{x}^*[n]}\}$$

$$\nabla\xi[\mathrm{n}] = -E\{e[n]\cdot \boldsymbol{x}^*[n]\}$$

- Thus, finally:

$$\boldsymbol{w}_{n+1} = \boldsymbol{w}_n + \mu\cdot E\{e[n]\cdot \boldsymbol{x}^*[n]\}$$

- So, the gradient is simplified by the average of the error $e[n]$ multiplied by the measured signal $\boldsymbol{x}^*[n]$

Convergence WSS

- If $d[n]$ and $x[n]$ are WSS, the gradient can be simplified as:

$$\nabla\xi[\mathrm{n}] = -E\{e[n]\cdot \boldsymbol{x}^*[n]\} = -(\underbrace{E\{d[n]\cdot \boldsymbol{x}^*[n]\}}_{r_{dx}} - \underbrace{E\{\boldsymbol{w}^T[n]\boldsymbol{x}[n]\cdot \boldsymbol{x}^*[n]\}}_{R_x\cdot w_n})$$

$$\nabla\xi[\mathrm{n}] = -(\boldsymbol{r}_{dx} - \boldsymbol{R}_x\cdot \boldsymbol{w}_n)$$

- The steepest descent algorithm become:

$$\boldsymbol{w}_{n+1} = \boldsymbol{w}_n + \mu\cdot(\boldsymbol{r}_{dx} - \boldsymbol{R}_x\cdot \boldsymbol{w}_n)$$

- Property: For WSS process $d[n]$ and $x[n]$, the steepest descent converge to Wiener-Hopf : $\lim_{n\to\infty} \boldsymbol{w}_n = \boldsymbol{R}_{xx}^{-1}\cdot \boldsymbol{r}_{dx}$

  if the step size is: $0 < \mu < \frac{2}{\lambda_{max}}$

  λ_{max} is the maximum eigenvalue of autocorrelation R_x so dependent of the signal amplitude!

LMS algorithm

- Previously, we saw the weight vector update for the steepest descent:

$$w_{n+1} = w_n + \mu \cdot E\{e[n] \cdot \boldsymbol{x}^*[n]\}$$

- Expectation $E\{\ \}$ is generally unknown, so it is replaced by:

$$\hat{E}\{e[n] \cdot \boldsymbol{x}^*[n]\} = \frac{1}{L}\sum_{k=0}^{L-1} e[n-k] \cdot \boldsymbol{x}[n-k]$$

- **LMS is simply using a single sample mean (L=1):**

$$\boldsymbol{w}_{n+1} = \boldsymbol{w}_n + \mu \cdot e[n] \cdot \boldsymbol{x}^*[n]$$

It is not as efficient as the averaging but much simpler and good enough for most of the cases.

LMS algorithm

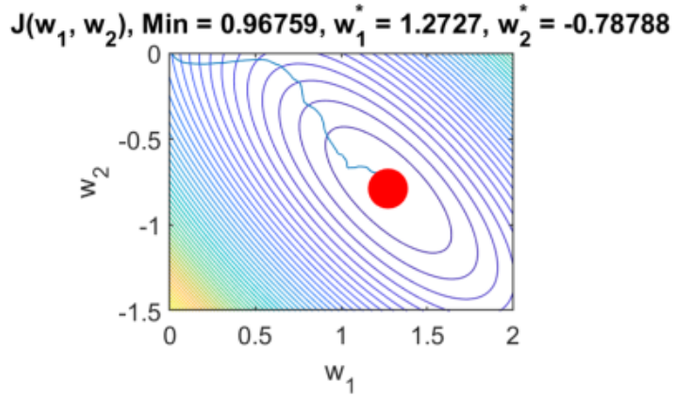
- The simplicity of the algorithm comes from the fact that the update for the *k*th coefficient:

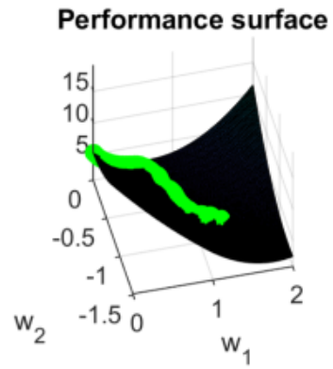
$$w_{n+1}[k] = w_n[k] + \mu \cdot e[n] \cdot x^*[n-k]$$

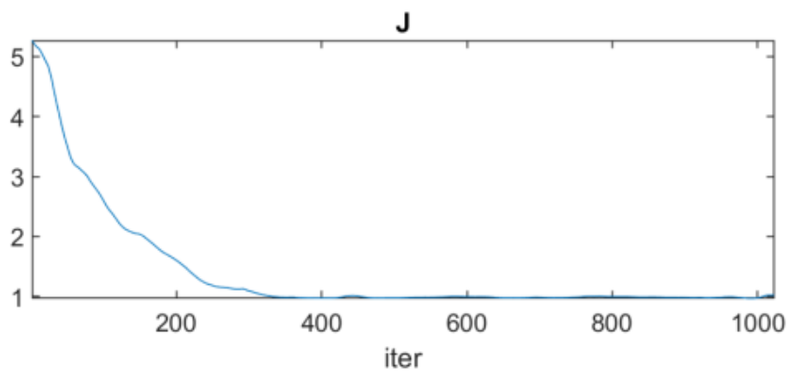
requires only one multiplication and one addition. The value for $\mu \cdot e[n]$ need only to be computed once and may be used for all the coefficients.

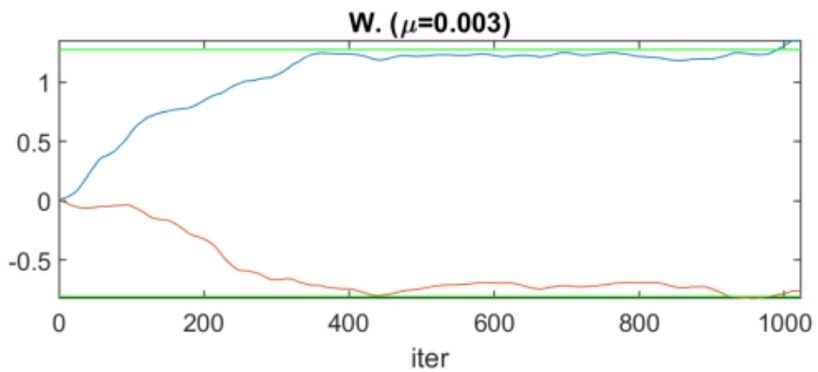
Example of convergence

Gradient descent (L = 30)

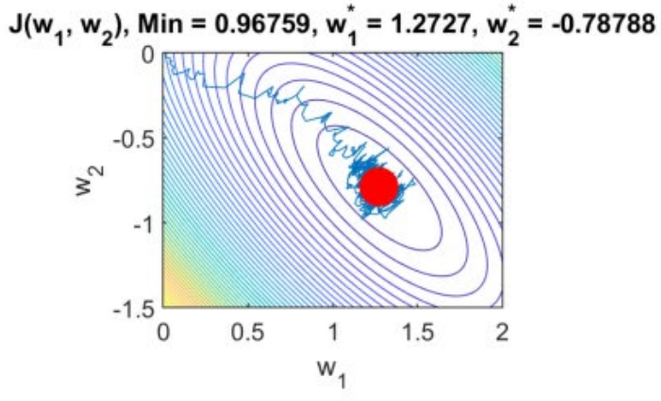


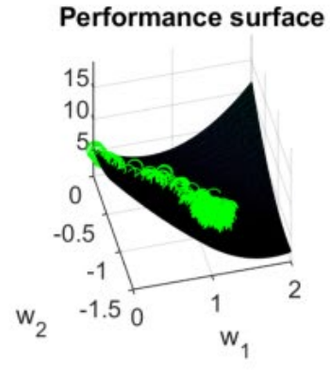


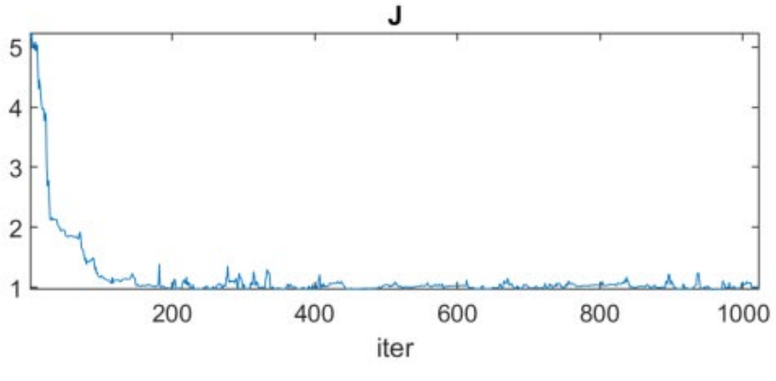


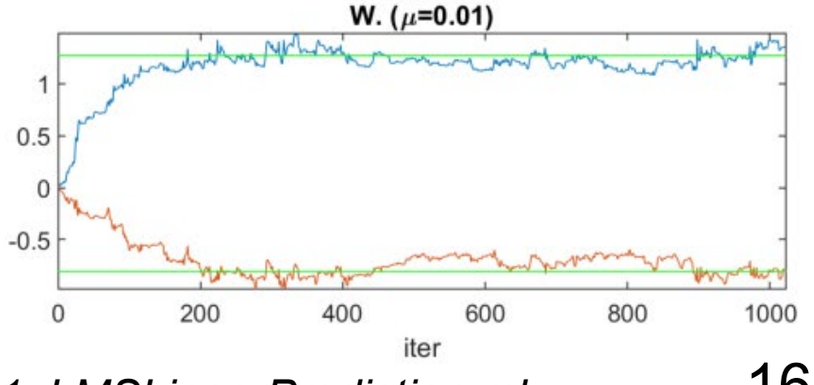


LMS (L = 1)









src : Ex9.2.1_LMSLinearPrediction.mlx

LMS with reduced complexity

- LMS algorithm to update weight can be even more optimized to reduce the calculation.

$$\boldsymbol{w}_{n+1} = \boldsymbol{w}_n + \mu \cdot \boxed{e[n] \cdot \boldsymbol{x}^*[n]}$$

- The product between the signal and the error is “time consuming” (not true today) because it is floating point. One simplification is **sign-error algorithm**:

$$\boldsymbol{w}_{n+1} = \boldsymbol{w}_n + \mu \cdot sign(e[n]) \cdot \boldsymbol{x}^*[n]$$

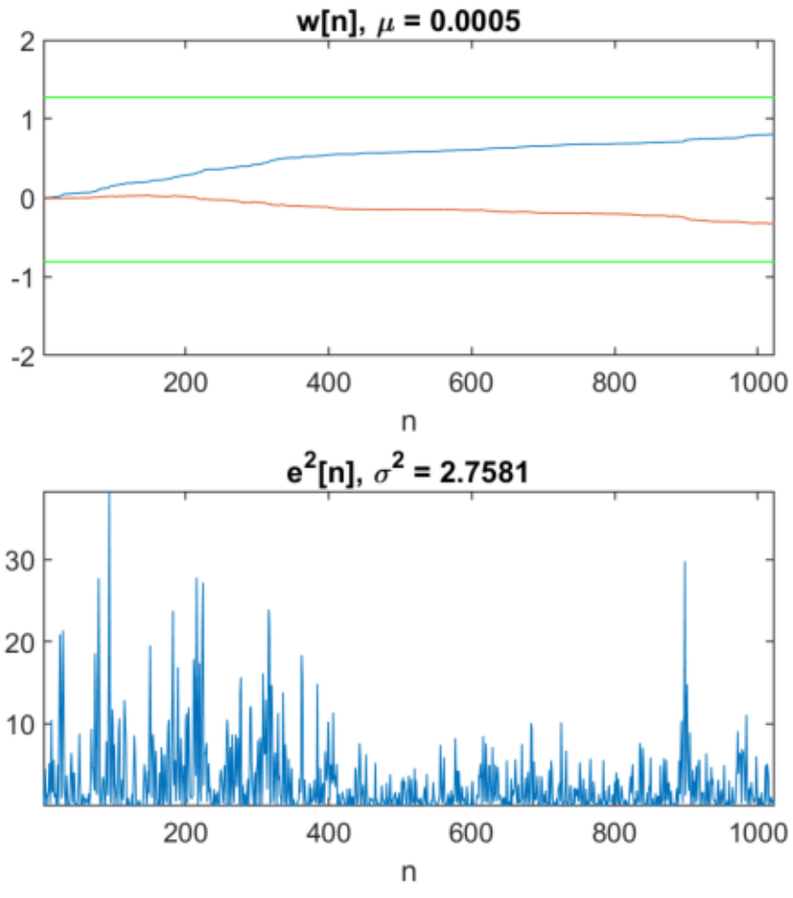
$$sign(e[n]) = \begin{cases} 1 & : e[n] > 0 \\ 0 & : e[n] = 0 \\ -1 & : e[n] < 0 \end{cases}$$

- Of course less efficient than LMS but faster to compute. Others simplifications exists as **sign-data** or **sign-sign.**

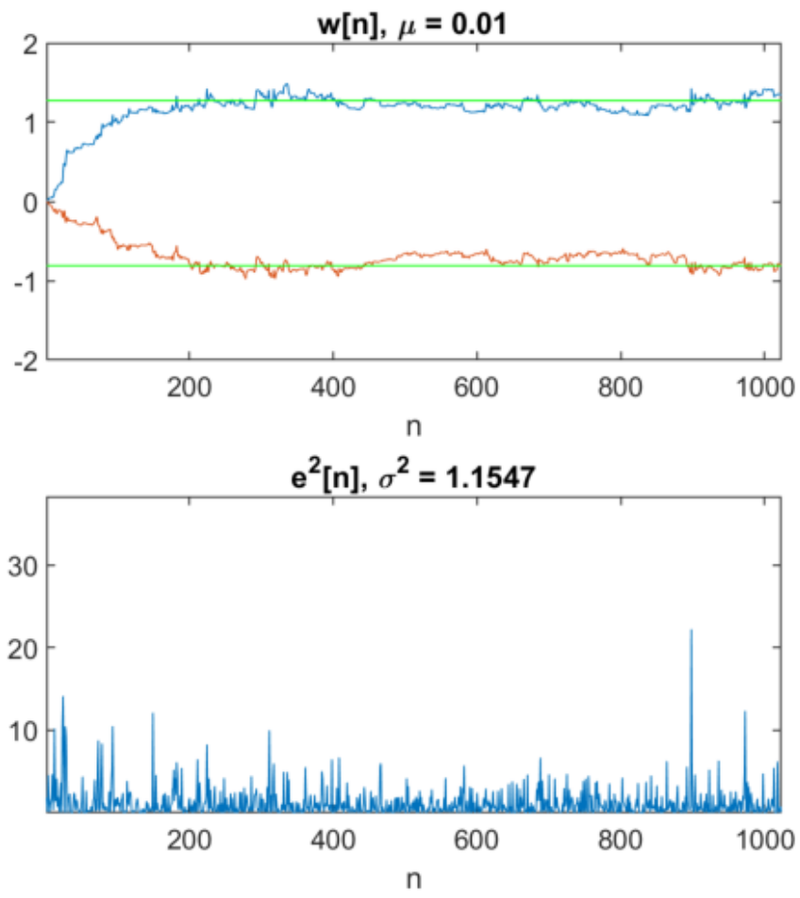
Normalized LMS

- One difficulty with LMS is the selection of the step size μ which define the rapidity of convergence.

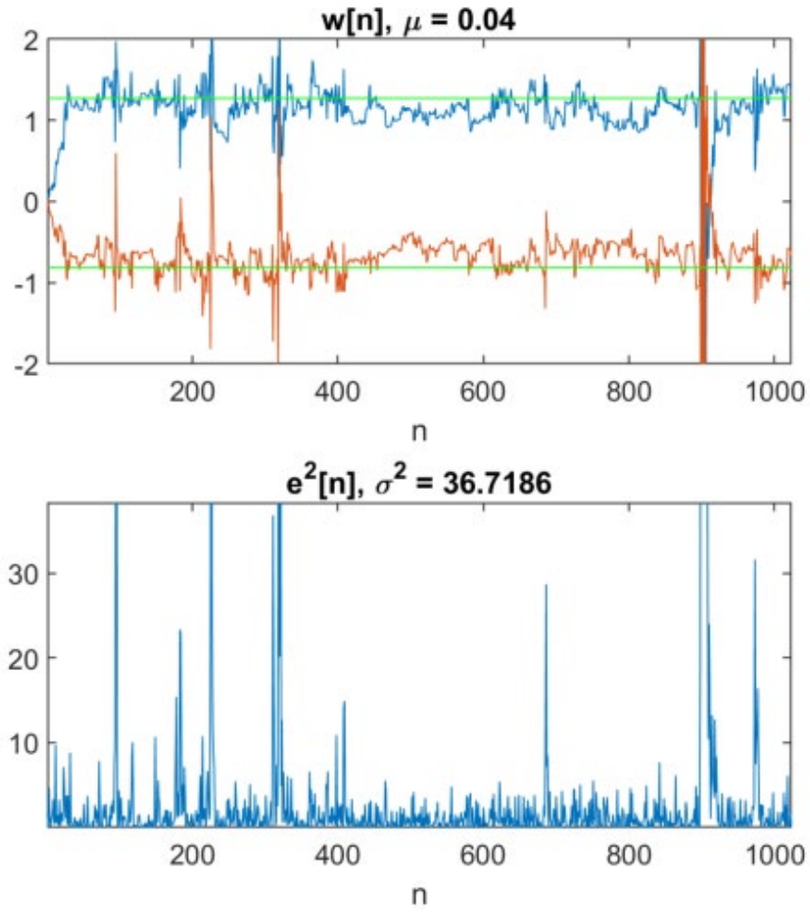
μ too small :



μ good :



μ too high :



- Unfortunately, this value is signal dependent. It will depend of its amplitude and variance. A solution is NLMS.

Normalized LMS algo

- The Normalized LMS algorithm called NLMS is given by:

$$\boldsymbol{w}_{n+1} = \boldsymbol{w}_n + \beta \cdot \frac{\boldsymbol{x}^*[n]}{\|\boldsymbol{x}[n]\|^2} \cdot e[n]$$

Where the signal is scaled by its squared norm.

- Where β is now bounded by : $0 < \beta < 2$
- If $\|x[n]\|$ is too small, we can use a constant $\varepsilon \approx 1 \cdot 10^{-4}$

$$\boldsymbol{w}_{n+1} = \boldsymbol{w}_n + \beta \cdot \frac{\boldsymbol{x}^*[n]}{\varepsilon + \|\boldsymbol{x}[n]\|^2} \cdot e[n]$$

- NLMS requires more calculation because of $\|\boldsymbol{x}[n]\|^2$ but can be evaluated recursively.

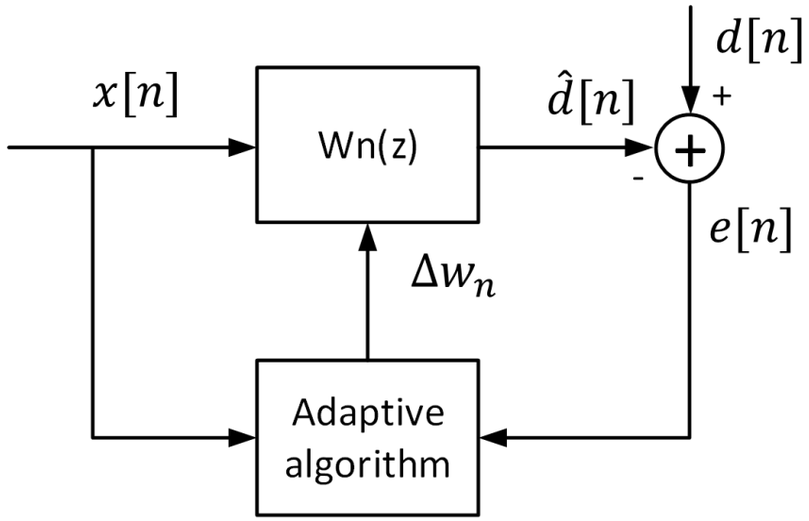
$$\|\boldsymbol{x}[n+1]\|^2 = \|\boldsymbol{x}[n]\|^2 + |\boldsymbol{x}[n+1]|^2 - |\boldsymbol{x}[n-p]|^2$$

Plan

- Introduction
- FIR Adaptive filter
- Recursive Least Square

Recursive Least Squares

- In previous adaptive filtering methods, the gradient descent is **minimizing the mean-square error**:



$$\xi[\mathrm{n}] = \mathrm{E}\{|e[n]|^2\}$$

$$\boldsymbol{w}_{n+1} = \boldsymbol{w}_n - \mu \cdot \nabla\xi[\mathrm{n}]$$

- The difficulty with these methods is that they all require knowledge of the autocorrelation of the input process $\boldsymbol{R}_x$ and the cross-correlation between the input and the desired output $\boldsymbol{r}_{dx}$

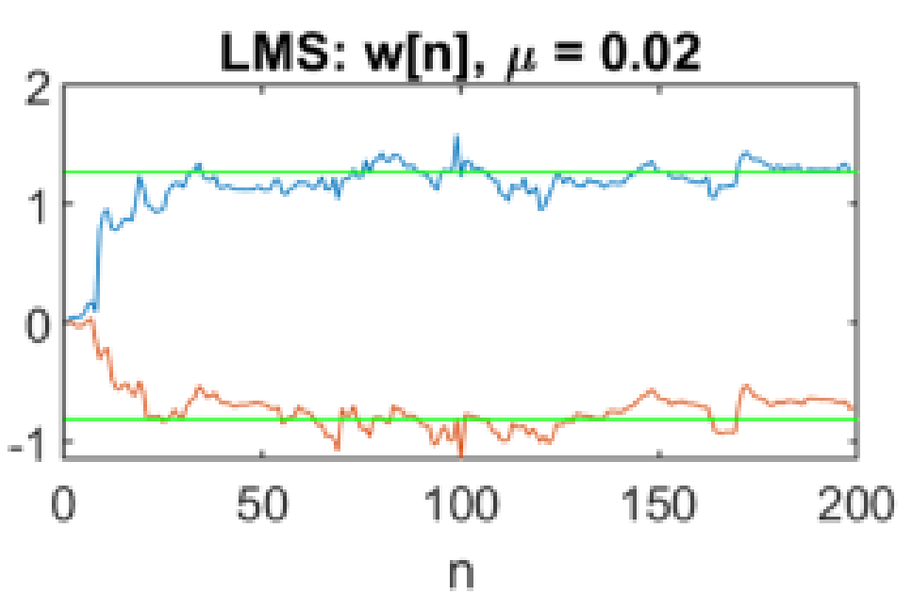
$$\nabla\xi[\mathrm{n}] = -E\{e[n] \cdot \boldsymbol{x}^*[n]\} = -(\underbrace{E\{d[n] \cdot \boldsymbol{x}^*[n]\}}_{\boldsymbol{r}_{dx}[\mathrm{n}]} - E\{\boldsymbol{w}^T[n]\underbrace{\boldsymbol{x}[n] \cdot \boldsymbol{x}^*[n]\}}_{\boldsymbol{R}_x[\mathrm{n}]})$$

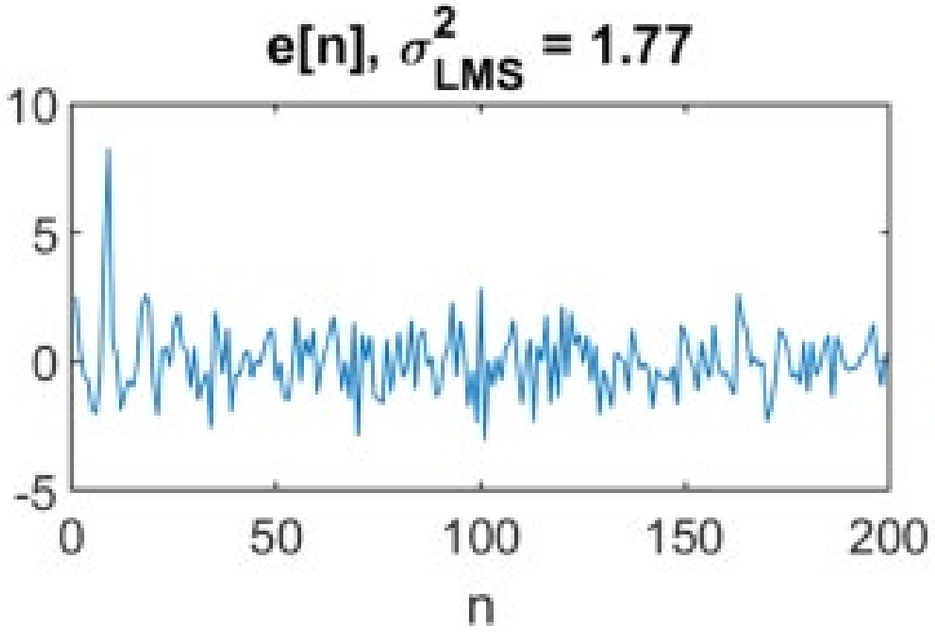
Recursive Least Squares

- When this statistical information is unknown, we have been forced to estimate these statistics from the data.
- In the LMS adaptive filter, these ensemble averages are **estimated** using instantaneous values :

$$\hat{E}\{e[n] \cdot \boldsymbol{x}^*[n]\} = e[n] \cdot \boldsymbol{x}^*\,[n]$$

- In some applications, this gradient may not converge rapidly or high excess mean-square error.





Recursive Least Squares

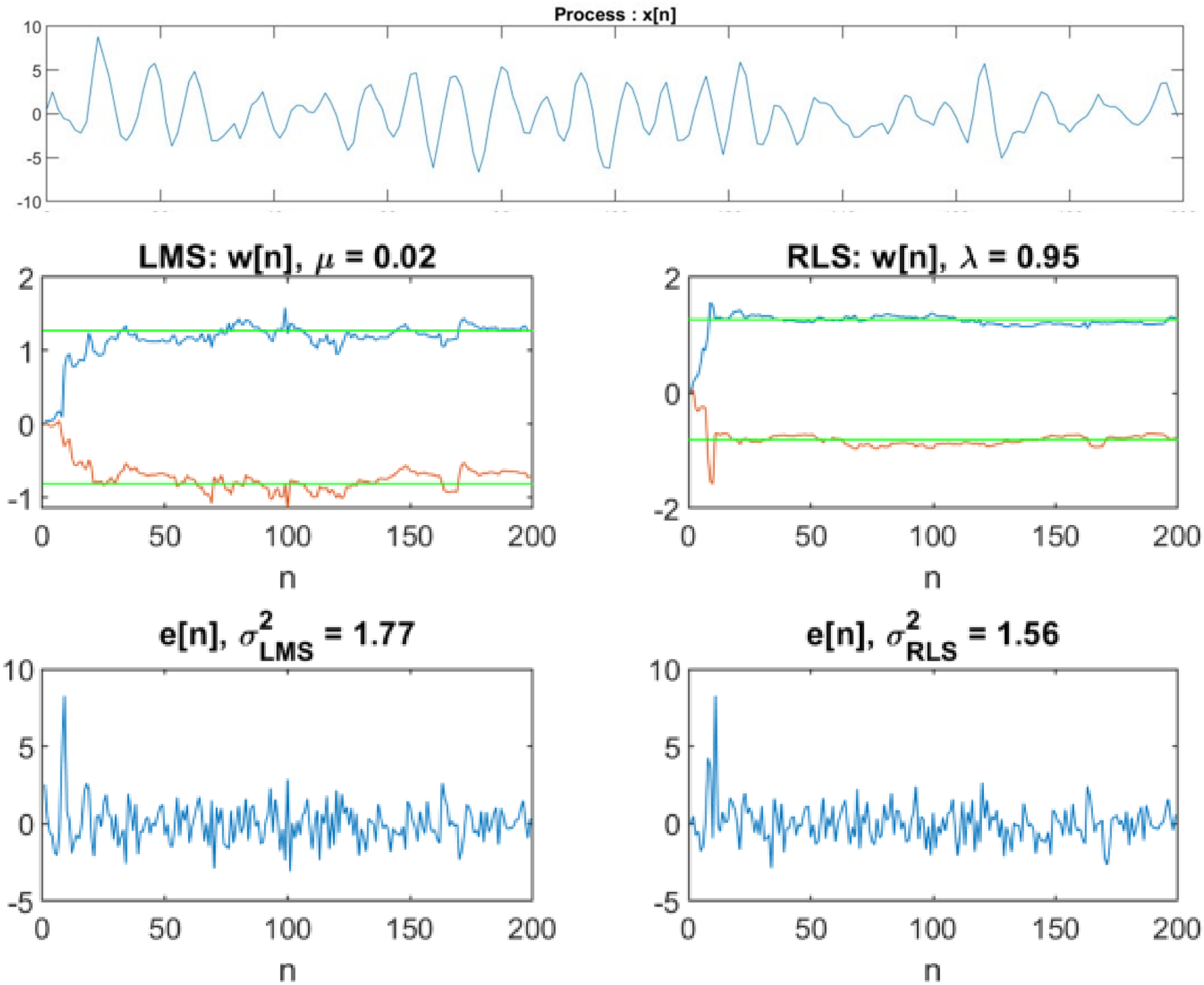
- An alternative is to consider error measures that do not include expectation and that maybe computed directly from the data. For example **minimizing a least-square error:**

$$\varepsilon[\mathrm{n}] = \sum_{i=0}^{n} |e[i]|^2$$

- Requires no statistical information about x[n] or d[n] and may be evaluated directly from x[n] and d[n]

Recursive Least Squares

- Can be slightly faster on stationary process than LMS



Exponentially weighted RLS

- Let us consider the design of a FIR filter

$$\boldsymbol{w}_n = \left[w_n[0], w_n[1], \cdots, w_n[p-1]\right]^T \qquad \boldsymbol{x}[n] = \left[x[n], x[n-1], \cdots, x[n-p-1]\right]^T$$

- On non-stationary process, the **last samples are more important than the previous**. To deal with it, the **exponentially weighted least square error** is developed:

$$\varepsilon[\mathrm{n}] = \sum_{i=0}^{n} \lambda^{n-i} |e[i]|^2 = \sum_{i=0}^{n} \lambda^{n-i} |d[i] - \boldsymbol{w}_n^T \cdot \boldsymbol{x}[i]|^2$$

Where $0 < \lambda \leq 1$ is an exponential weighting (forgetting factor).

Exponentially weighted RLS

- To find the coefficients, we proceed in the same way than for LMS:

$$\frac{\partial \varepsilon[\mathrm{n}]}{\partial w_n^*[k]} = 0 = \sum_{i=0}^{n} \lambda^{n-i} e[i] \frac{\partial e[i]}{\partial w_n^*[k]} = \sum_{i=0}^{n} \lambda^{n-i} e[i] x^*[i-k]$$

$$= \sum_{i=0}^{n} \lambda^{n-i} \left\{ d[i] - \sum_{l=0}^{p} w_n[l] x[i-l] \right\} x^*[i-k]$$

$$\Rightarrow \sum_{l=0}^{p-1} w_n[l] \underbrace{\left[\sum_{i=0}^{n} \lambda^{n-i} x[i-l] x^*[i-k] \right]}_{\boldsymbol{R_x}[n]} = \underbrace{\sum_{i=0}^{n} \lambda^{n-i} d[i] x^*[i-k]}_{\boldsymbol{r_{dx}}[n]}$$

$$\boldsymbol{R_x}[n] = \sum_{i=0}^{n} \lambda^{n-i} \boldsymbol{x}^*[i] \boldsymbol{x}^T[i]$$

[p x p]

$$\boldsymbol{r_{dx}}[n] = \sum_{i=0}^{n} \lambda^{n-i} d[i] \boldsymbol{x}^*[i]$$

[p x 1]

$$\frac{\partial \varepsilon[\mathrm{n}]}{\partial w_n^*[k]} = 0 \Rightarrow \boxed{\boldsymbol{R_x}[n] \cdot \boldsymbol{w_n} = \boldsymbol{r_{dx}}[n]}$$

Exponentially weighted RLS

- Since $\boldsymbol{R}_x[n]$ and $\boldsymbol{r}_{dx}[n]$ depend on n, instead of solving the deterministic normal equations, we will derive a recursion solution:

$$\boldsymbol{w}_n = \boldsymbol{w}_{n-1} + \Delta\boldsymbol{w}_{n-1}$$

Where $\Delta\boldsymbol{w}_{n-1}$ is a correction applied to solution at time n-1. Since:

$$\boldsymbol{w}_n = \boldsymbol{R}_x^{-1}[n]\,\boldsymbol{r}_{dx}[n]$$

The recursion is obtained by expressing $\boldsymbol{r}_{dx}[n]$ in terms of $\boldsymbol{r}_{dx}[n-1]$ and $\boldsymbol{R}_x^{-1}[n]$ in terms of $\boldsymbol{R}_x^{-1}[n-1]$ and the new data vector $\boldsymbol{x}\,[n]$:

$$\boldsymbol{r}_{\boldsymbol{dx}}[n] = \lambda\boldsymbol{r}_{\boldsymbol{dx}}[n-1] + d[n]\cdot\boldsymbol{x}^*[n]$$

$$\boldsymbol{R}_x[n] = \lambda\boldsymbol{R}_x[n-1] + \boldsymbol{x}^*[n]\cdot\boldsymbol{x}^T[n]$$

Woodbury's identity

- This mathematical Woodbury identity (eq 2.29, p.29, Stat. Dig., Monson H.Hayes). is the central element in RLS efficiency. It can update the inverse covariance matrix without computing the complete inversion at each iteration :

$$(A + uv^T)^{-1} = A^{-1} - \frac{A^{-1}uv^T A^{-1}}{1 + v^T A^{-1} u}$$

$$\boldsymbol{R}_x^{-1}[n] = (\underbrace{\lambda \boldsymbol{R}_x[n-1]}_{A} + \underbrace{\boldsymbol{x}^*[n]}_{u} \cdot \underbrace{\boldsymbol{x}^T[n]}_{v^T})^{-1}$$

To simplify the notation, we write : $\boldsymbol{P}[n] = \boldsymbol{R}_x^{-1}[n]$

$$\boldsymbol{R}_x^{-1}[n] = \boldsymbol{P}[n] = \lambda^{-1}\boldsymbol{P}[n-1] - \frac{\lambda^{-1}\boldsymbol{P}[n-1]x[n]x^T[n]\lambda^{-1}\boldsymbol{P}[n-1]}{1 + x^T[n]\lambda^{-1}\boldsymbol{P}[n-1]}$$

$$\Rightarrow \quad \boldsymbol{R}_x^{-1}[n] = \boldsymbol{P}[n] = \lambda^{-1}\left[\boldsymbol{P}[n-1] - \frac{\lambda^{-1}\boldsymbol{P}[n-1]x[n]}{1 + x^T[n]\lambda^{-1}\boldsymbol{P}[n-1]} x^T[n]\, \boldsymbol{P}[n-1]\right]$$

Exponentially weighted RLS

$$\boldsymbol{g}[n] = \frac{\lambda^{-1}\boldsymbol{P}[n-1]\cdot\boldsymbol{x}^*[n]}{1+\lambda^{-1}\boldsymbol{x}^T[n]\boldsymbol{P}[n-1]\cdot\boldsymbol{x}^*[n]}$$
$$\boldsymbol{P}[n] = \lambda^{-1}\left[\boldsymbol{P}[n-1] - \boldsymbol{g}[n]\cdot\boldsymbol{x}^T[n]\boldsymbol{P}[n-1]\right]$$

- And finally, the filter coefficients are:

$$\boldsymbol{w}_n = \boldsymbol{R}_x^{-1}[n]\boldsymbol{r}_{\boldsymbol{dx}}[n]$$

- By replacing $\boldsymbol{R}_x^{-1}[n]$ and $\boldsymbol{r}_{\boldsymbol{dx}}[n]$ by the recursive version :

$$\boldsymbol{R}_x^{-1}[n] = \boldsymbol{P}[n] \text{ and } \boldsymbol{r}_{\boldsymbol{dx}}[n] = \lambda\boldsymbol{r}_{\boldsymbol{dx}}[n-1] + d[n]\cdot\boldsymbol{x}^*[n]$$

- The filter coefficients are then computed :

$$\boldsymbol{w}_n = \lambda\boldsymbol{P}[n]\boldsymbol{r}_{\boldsymbol{dx}}[n-1] + d[n]\boldsymbol{P}[n]\boldsymbol{x}^*[n]$$

- By using the previous results of $\boldsymbol{P}[n]$ and $\boldsymbol{g}[n]$

$$\boldsymbol{w}_n = \left[\boldsymbol{P}[n-1] - \boldsymbol{g}[n]\boldsymbol{x}^T[n]\boldsymbol{P}[n-1]\right]\boldsymbol{r}_{\boldsymbol{dx}}[n-1] + d[n]\boldsymbol{g}[n]$$

Exponentially weighted RLS

- Finally, recognizing that $\boldsymbol{P}[n-1]\boldsymbol{r_{dx}}[n-1] = \boldsymbol{w}_{n-1}$ it follows that

$$\boldsymbol{w}_n = \boldsymbol{w}_{n-1} + \boldsymbol{g}[n]\left[d[n] - \boldsymbol{w}_{n-1}^{\boldsymbol{T}}\boldsymbol{x}[n]\right]$$

- Which may be written as

$$\Rightarrow \quad \boldsymbol{w}_n = \boldsymbol{w}_{n-1} + \alpha[n] \cdot \boldsymbol{g}[n]$$

Where:

$\alpha[n]$ is the a priori error (using previous coeff.): $\alpha[n] = d[n] - \boldsymbol{w}_{n-1}^{\boldsymbol{T}}\boldsymbol{x}[\boldsymbol{n}]$

$\boldsymbol{g}[n]$ is the gain vector

MASTER OF SCIENCE IN ENGINEERING

Exponentially weighted RLS implementation

Parameters:

- p : number of coefficient (filter order +1)
- λ : Exponential weighting factor
- δ : value used to initialize **P**[0]

Initialization:

- $\boldsymbol{w}_0 = \mathbf{0}$
- $\boldsymbol{P}[0] = \delta^{-1}\boldsymbol{I}$

Computation:

For n = 1,2,… compute

$$\mathbf{z}[n] = \boldsymbol{P}[n-1] \cdot \boldsymbol{x}^*[n] \qquad [p\ x\ 1] = [p\ x\ p]\ [p\ x\ 1]$$

#gain vector

$$\boldsymbol{g}[n] = \frac{\boldsymbol{z}[\boldsymbol{n}]}{\lambda + \boldsymbol{x}^T[n]\boldsymbol{z}[\boldsymbol{n}]} \qquad [p\ x\ 1] = \frac{[p\ x\ 1]}{[1\ x\ 1] + [1\ x\ p][p\ x\ 1]}$$

#a priori error

$$\alpha[n] = d[n] - \boldsymbol{w}_{n-1}^{\boldsymbol{T}}\boldsymbol{x}[n] \qquad [1\ x\ 1] = [1\ x\ 1] - [1\ x\ p][p\ x\ 1]$$

#filter coefficient update

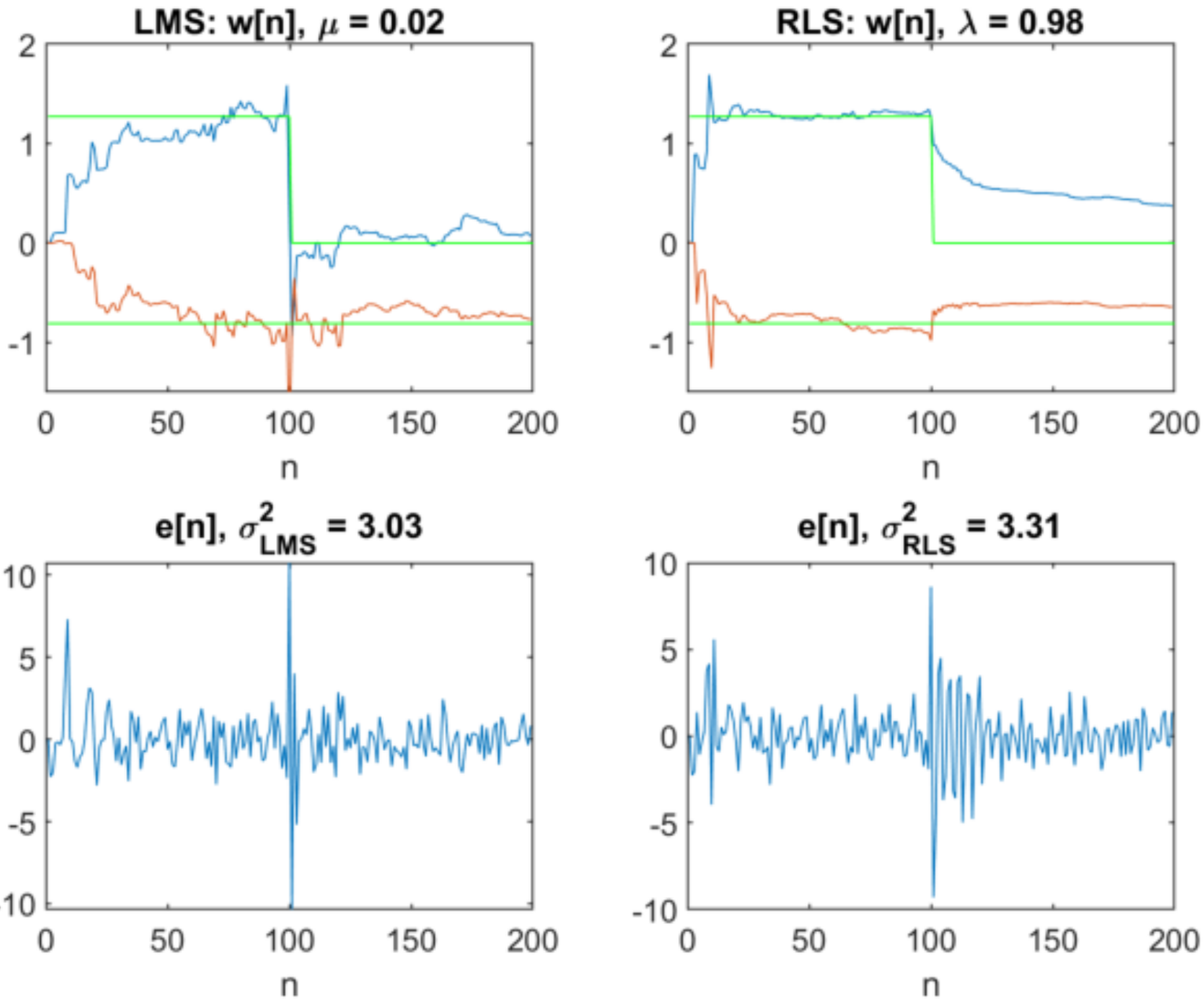
$$\boldsymbol{w}_n = \boldsymbol{w}_{n-1} + \alpha[n]\boldsymbol{g}[n] \qquad [p\ x\ 1] = [p\ x\ 1] + [1\ x\ 1]\ [p\ x\ 1]$$

#inverse autocorrelation matrix update

$$\boldsymbol{P}[n] = \frac{1}{\lambda}\left[\boldsymbol{P}[n-1] - \boldsymbol{g}[n] \cdot \boldsymbol{z}^{\boldsymbol{T}}[n]\right] \qquad [p\ x\ p] = [p\ x\ p] - [p\ x\ 1][1\ x\ p]$$

Example Linear Prediction non stationary process example (9.4.2)

- Comparison between LMS and RLS for non-stationary process



LMS versus RLS

- This table summarizes the key differences between the two types of algorithms:

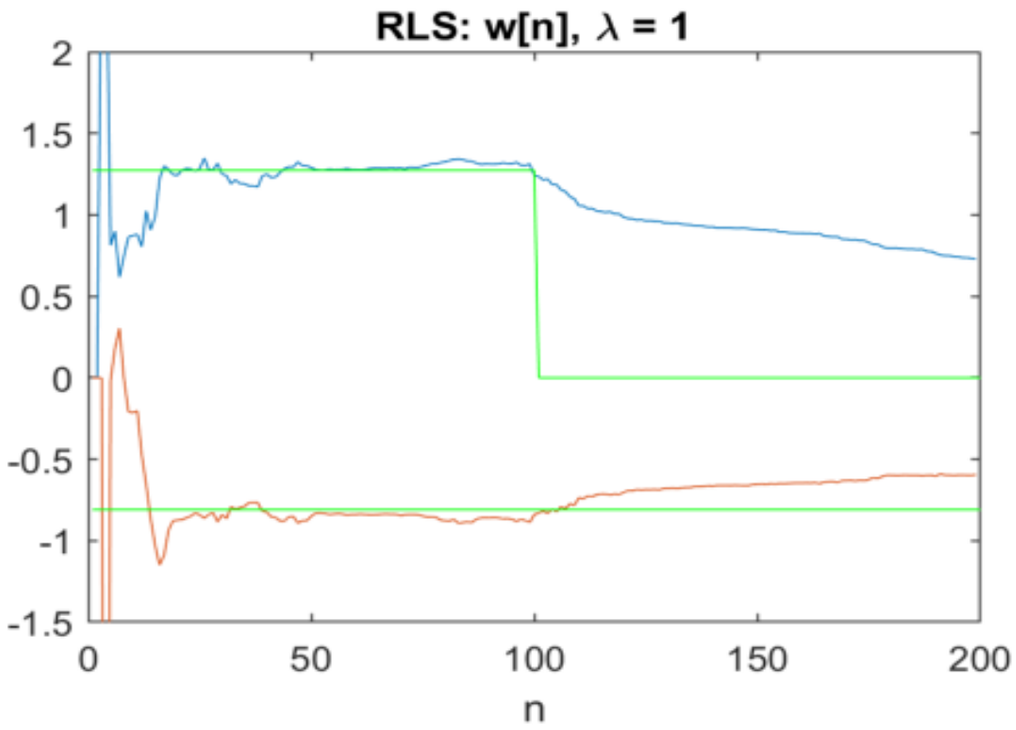
LMS algorithm	RLS algorithm
Simple and can be easily applied	**Increased complexity and computational cost**
Takes longer to converge	**Faster to converge**
Adaptation is based on the gradient-based approach that updates filter weights to converge to the minimum filter weights	Adaptation is based on the recursive approach that finds the filter coefficients that minimize a weighted linear least squares cost function relating to the input signals.
Larger steady state error with respect to the unknown system.	Smaller steady state error with respect to unknown system.
Does not account for past data.	**Accounts for past data from the beginning to the current data point.**
Objective is to minimize the current mean square error between the desired signal and the output.	Objective is to minimize the total weighted squared error between the desired signal and the output.
No memory involved. Older error values play no role in the total error considered.	Has infinite memory. All error data is considered in the total error. Using the forgetting factor, the older data can be de-emphasized compared to the newer data. Since $0 \leq \lambda < 1$, applying the factor is equivalent to weighting the older error.

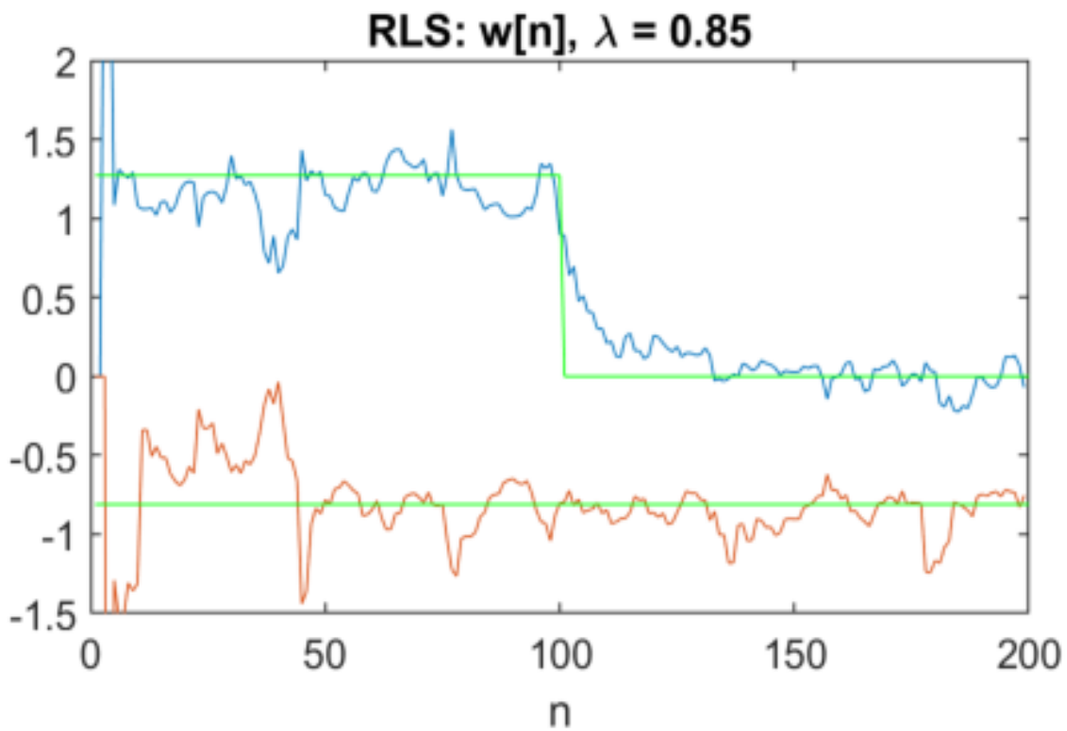
src : Mathworks

Sliding window RLS

- **When $\lambda = 1$,** RLS algo has a growing window and each $|e[i]|^2, i = 0, \dots, n$ are equally weighted, n being the current time index
- **When $\lambda < 1$,** $|e[i]|^2$ become less important form values of i that are small compared to n (forgetting factor).
- In both case, RLS algo **requires infinite memory** in the sense that all the data beginning from time n=0 will affect the coefficients w[n]

Example of non-stationary process where $\lambda = 1$ is not able to track.





src : figure926_*.mlx

Sliding window RLS

- **For non-stationary process**, a small value of λ (small horizon) is necessary. Or minimizing the squared error on a finite length : L

$$\varepsilon[\mathrm{n}] = \sum_{i=0}^{n} \lambda^{n-i} |e[i]|^2 \quad \Rightarrow \quad \varepsilon[\mathrm{n}] = \sum_{i=n-L}^{n} |e[i]|^2$$

RLS — Sliding RLS

- If L= 0 → it results to LMS algorithm
- This small modification by introducing L is adding more complexity in the calculations than RLS (see next page).

- It requires about **twice the number of multiplications** and additions, and requires p+L values of x[n] to be stored.

Sliding window RLS

- The sliding window is modifying the update equation:

Step 1

$$\boldsymbol{g}[n+1] = \frac{\boldsymbol{P}[n-1]\boldsymbol{x}^*[n]}{1 + \boldsymbol{x}^T[n]\boldsymbol{P}[n-1]\boldsymbol{x}[n]}$$

$$\widetilde{\boldsymbol{w}}_n = \boldsymbol{w}_{n-1} + \boldsymbol{g}[n] \cdot \left[d[n] - \boldsymbol{w}_{n-1} \cdot \boldsymbol{x}^*[n]\right]$$

$$\widetilde{\boldsymbol{P}}[n+1] = \boldsymbol{P}[n-1] - \boldsymbol{g}[n]\boldsymbol{x}^T[n]\boldsymbol{P}[n-1]$$

Step 2

$$\widetilde{\boldsymbol{g}}[n+1] = \frac{\widetilde{\boldsymbol{P}}[n]\boldsymbol{x}^*[n-L-1]}{1 - \boldsymbol{x}^T[n-L-1]\tilde{P}[n]\boldsymbol{x}[n-L-1]}$$

$$\boldsymbol{w}_n = \widetilde{\boldsymbol{w}}_n - \widetilde{\boldsymbol{g}}[n] \cdot \left[d[n-L-1] - \widetilde{\boldsymbol{w}}_n \cdot \boldsymbol{x}^*[n-L-1]\right]$$

$$\boldsymbol{P}[n] = \widetilde{\boldsymbol{P}}[n] + \widetilde{\boldsymbol{g}}[n]\boldsymbol{x}^T[n-L-1]\widetilde{\boldsymbol{P}}[n]$$